

```
# Customer Segmentation Analysis
# The aim of this data analytics project is to perform customer segmentation analysis for an e-
# commerce company. By analyzing customer behavior and purchase patterns, the goal is to
# group customers into distinct segments. This segmentation can inform targeted marketing
# strategies, improve customer satisfaction, and enhance overall business strategies.
```

```
# import libraries to use
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# load csv file
from google.colab import files
uploaded = files.upload()
```

Choose files ifood_df.csv

ifood_df.csv(text/csv) - 219474 bytes, last modified: 23/06/2026 - 100% done

Saving ifood_df.csv to ifood_df.csv

```
df = pd.read_csv('ifood_df.csv')
df.head()
```

	Income	Kidhome	Teenhome	Recency	MntWines	MntFruits	MntMeatProducts	MntFishProducts	MntSweetProducts	MntGoldProds
0	58138.0	0	0	58	635	88	546	172	88	88
1	46344.0	1	1	38	11	1	6	2	1	6
2	71613.0	0	0	26	426	49	127	111	21	42
3	26646.0	1	0	26	11	4	20	10	3	5
4	58293.0	1	0	94	173	43	118	46	27	15

5 rows × 39 columns

```
df.shape
```

```
(2205, 39)
```

```
# Data Collection: Obtain a dataset containing customer information, purchase history, and relevant data.
df.describe()
```

	Income	Kidhome	Teenhome	Recency	MntWines	MntFruits	MntMeatProducts	MntFishProducts	MntSweetProducts
count	2205.000000	2205.000000	2205.000000	2205.000000	2205.000000	2205.000000	2205.000000	2205.000000	2205.000000
mean	51622.094785	0.442177	0.506576	49.009070	306.164626	26.403175	165.312018	37.756463	21.000000
std	20713.063826	0.537132	0.544380	28.932111	337.493839	39.784484	217.784507	54.824635	15.000000
min	1730.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	35196.000000	0.000000	0.000000	24.000000	24.000000	2.000000	16.000000	3.000000	0.000000
50%	51287.000000	0.000000	0.000000	49.000000	178.000000	8.000000	68.000000	12.000000	0.000000
75%	68281.000000	1.000000	1.000000	74.000000	507.000000	33.000000	232.000000	50.000000	0.000000
max	113734.000000	2.000000	2.000000	99.000000	1493.000000	199.000000	1725.000000	259.000000	0.000000

8 rows × 39 columns

```
df.columns.tolist() #checks cols name
```

```
['Income',
 'Kidhome',
 'Teenhome',
 'Recency',
 'MntWines',
 'MntFruits',
 'MntMeatProducts',
 'MntFishProducts',
 'MntSweetProducts',
 'MntGoldProds']
```

```
'MntGoldProds',
'NumDealsPurchases',
'NumWebPurchases',
'NumCatalogPurchases',
'NumStorePurchases',
'NumWebVisitsMonth',
'AcceptedCmp3',
'AcceptedCmp4',
'AcceptedCmp5',
'AcceptedCmp1',
'AcceptedCmp2',
'Complain',
'Z_CostContact',
'Z_Revenue',
'Response',
'Age',
'Customer_Days',
'marital_Divorced',
'marital_Married',
'marital_Single',
'marital_Together',
'marital_Widow',
'education_2n Cycle',
'education_Basic',
'education_Graduation',
'education_Master',
'education_PhD',
'MntTotal',
'MntRegularProds',
'AcceptedCmpOverall']
```

```
# Data Exploration and Cleaning:
# Explore the dataset, understand its structure, and handle any missing or inconsistent data.
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2205 entries, 0 to 2204
Data columns (total 39 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Income                2205 non-null  float64
1   Kidhome               2205 non-null  int64
2   Teenhome              2205 non-null  int64
3   Recency               2205 non-null  int64
4   MntWines              2205 non-null  int64
5   MntFruits             2205 non-null  int64
6   MntMeatProducts       2205 non-null  int64
7   MntFishProducts       2205 non-null  int64
8   MntSweetProducts      2205 non-null  int64
9   MntGoldProds          2205 non-null  int64
10  NumDealsPurchases     2205 non-null  int64
11  NumWebPurchases       2205 non-null  int64
12  NumCatalogPurchases   2205 non-null  int64
13  NumStorePurchases     2205 non-null  int64
14  NumWebVisitsMonth     2205 non-null  int64
15  AcceptedCmp3          2205 non-null  int64
16  AcceptedCmp4          2205 non-null  int64
17  AcceptedCmp5          2205 non-null  int64
18  AcceptedCmp1          2205 non-null  int64
19  AcceptedCmp2          2205 non-null  int64
20  Complain              2205 non-null  int64
21  Z_CostContact          2205 non-null  int64
22  Z_Revenue             2205 non-null  int64
23  Response              2205 non-null  int64
24  Age                   2205 non-null  int64
25  Customer_Days         2205 non-null  int64
26  marital_Divorced      2205 non-null  int64
27  marital_Married       2205 non-null  int64
28  marital_Single        2205 non-null  int64
29  marital_Together      2205 non-null  int64
30  marital_Widow         2205 non-null  int64
31  education_2n Cycle    2205 non-null  int64
32  education_Basic       2205 non-null  int64
33  education_Graduation  2205 non-null  int64
34  education_Master      2205 non-null  int64
35  education_PhD         2205 non-null  int64
36  MntTotal              2205 non-null  int64
37  MntRegularProds       2205 non-null  int64
38  AcceptedCmpOverall    2205 non-null  int64
dtypes: float64(1), int64(38)
memory usage: 672.0 KB
```

```
df.isnull().sum()
# Observation -> No missing Values found
```

	0
Income	0
Kidhome	0
Teenhome	0
Recency	0
MntWines	0
MntFruits	0
MntMeatProducts	0
MntFishProducts	0
MntSweetProducts	0
MntGoldProds	0
NumDealsPurchases	0
NumWebPurchases	0
NumCatalogPurchases	0
NumStorePurchases	0
NumWebVisitsMonth	0
AcceptedCmp3	0
AcceptedCmp4	0
AcceptedCmp5	0
AcceptedCmp1	0
AcceptedCmp2	0
Complain	0
Z_CostContact	0
Z_Revenue	0
Response	0
Age	0
Customer_Days	0
marital_Divorced	0
marital_Married	0
marital_Single	0
marital_Together	0
marital_Widow	0
education_2n Cycle	0
education_Basic	0
education_Graduation	0
education_Master	0
education_PhD	0
MntTotal	0
MntRegularProds	0
AcceptedCmpOverall	0

dtype: int64

```
df.duplicated().sum() #checks duplicate -> 184 values were duplicate
```

```
np.int64(184)
```

```
df.drop_duplicates(inplace=True) #remove duplicates
```

```
df.duplicated().sum() # hence duplicate values are deleted
```

```
np.int64(0)
```

df.dtypes

	0
Income	float64
Kidhome	int64
Teenhome	int64
Recency	int64
MntWines	int64
MntFruits	int64
MntMeatProducts	int64
MntFishProducts	int64
MntSweetProducts	int64
MntGoldProds	int64
NumDealsPurchases	int64
NumWebPurchases	int64
NumCatalogPurchases	int64
NumStorePurchases	int64
NumWebVisitsMonth	int64
AcceptedCmp3	int64
AcceptedCmp4	int64
AcceptedCmp5	int64
AcceptedCmp1	int64
AcceptedCmp2	int64
Complain	int64
Z_CostContact	int64
Z_Revenue	int64
Response	int64
Age	int64
Customer_Days	int64
marital_Divorced	int64
marital_Married	int64
marital_Single	int64
marital_Together	int64
marital_Widow	int64
education_2n Cycle	int64
education_Basic	int64
education_Graduation	int64
education_Master	int64
education_PhD	int64
MntTotal	int64
MntRegularProds	int64
AcceptedCmpOverall	int64

dtype: object

```
# Descriptive Statistics:
# Calculate key metrics such as average purchase value, frequency of purchases, etc.
df.describe()
```

	Income	Kidhome	Teenhome	Recency	MntWines	MntFruits	MntMeatProducts	MntFishProducts	MntSwe
count	2021.000000	2021.000000	2021.000000	2021.000000	2021.000000	2021.000000	2021.000000	2021.000000	2021.000000
mean	51687.258783	0.443345	0.509649	48.880752	306.492331	26.364671	166.059871	37.603662	37.603662
std	20713.046401	0.536196	0.546393	28.950917	337.603877	39.776518	219.869126	54.892196	54.892196
min	1730.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	35416.000000	0.000000	0.000000	24.000000	24.000000	2.000000	16.000000	3.000000	3.000000
50%	51412.000000	0.000000	0.000000	49.000000	178.000000	8.000000	68.000000	12.000000	12.000000
75%	68274.000000	1.000000	1.000000	74.000000	507.000000	33.000000	230.000000	50.000000	50.000000
max	113734.000000	2.000000	2.000000	99.000000	1493.000000	199.000000	1725.000000	259.000000	259.000000

8 rows × 39 columns

```
df['Income'].mean() #Average Customer Income
```

```
np.float64(51687.2587827808)
```

```
df['MntTotal'].mean() #Avg Customer spending
```

```
np.float64(563.789213260762)
```

```
df['Recency'].mean() #Average Recency
```

```
np.float64(48.880752102919345)
```

```
# Average Purchases
```

```
df['TotalPurchases'] = (df['NumWebPurchases']+df['NumCatalogPurchases']+ df['NumStorePurchases'])
```

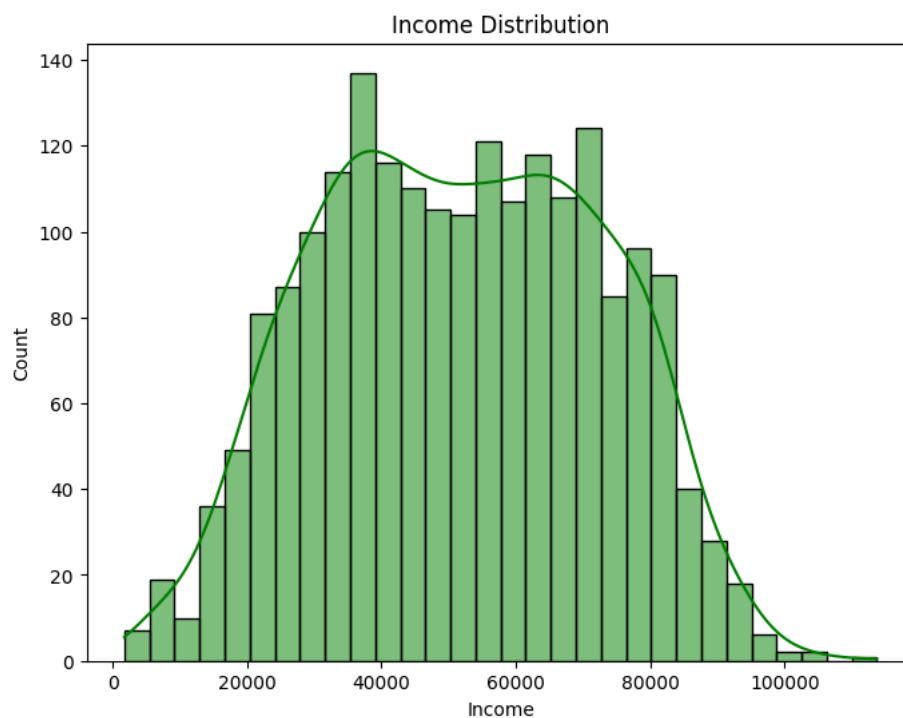
```
df['TotalPurchases'].mean()
```

```
np.float64(12.567540821375557)
```

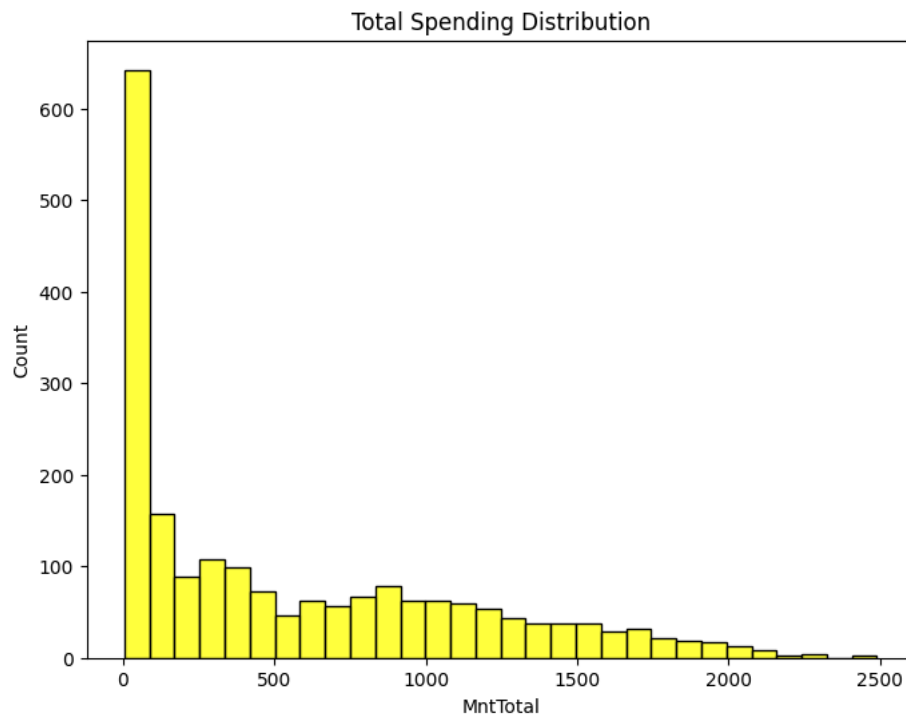
```
# Exploratory Data Analysis
```

```
# income distribution
```

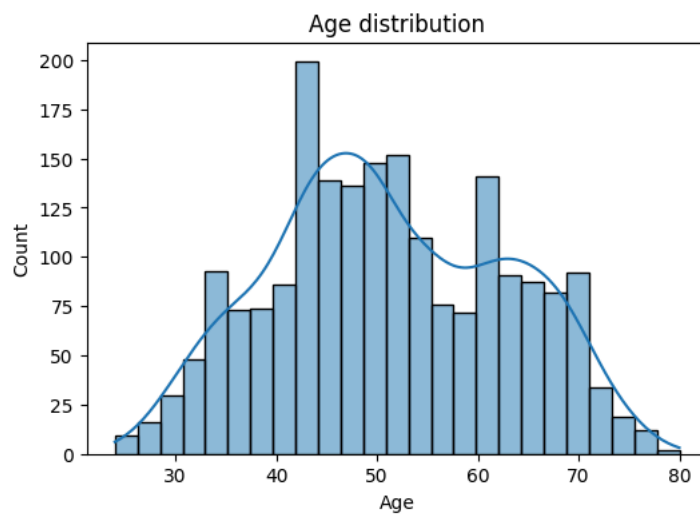
```
plt.figure(figsize=(8,6))
sns.histplot(df['Income'], bins=30,kde = True, color = 'green')
plt.title("Income Distribution")
plt.show()
```



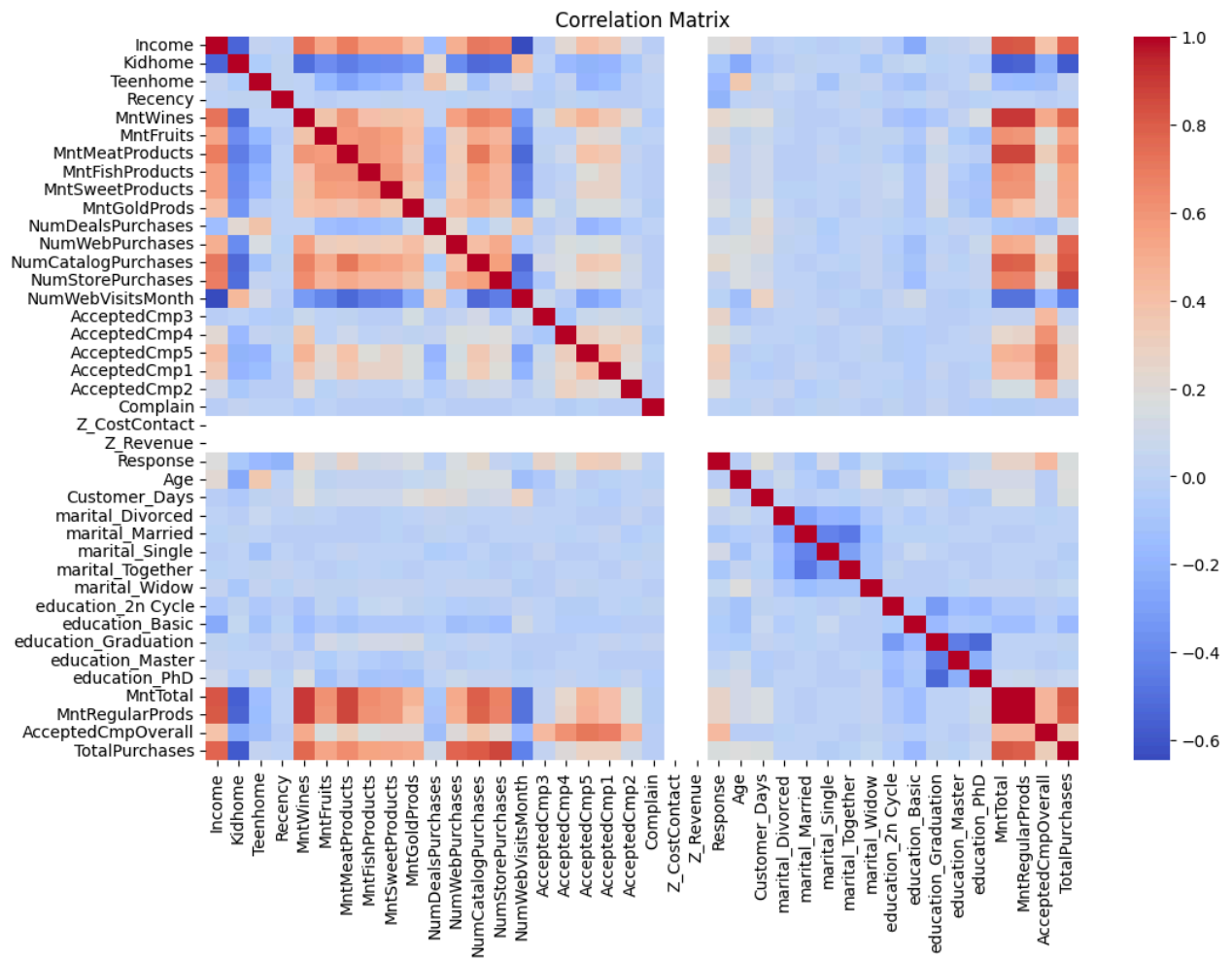
```
plt.figure(figsize=(8,6))
sns.histplot(df['MntTotal'], bins=30, color = 'yellow')
plt.title("Total Spending Distribution")
plt.show()
```



```
plt.figure(figsize=(6,4))
sns.histplot(df['Age'], bins= 25,kde = True)
plt.title('Age distribution')
plt.show()
```



```
plt.figure(figsize=(12,8))
sns.heatmap(df.corr(numeric_only=True),cmap = 'coolwarm', annot = False)
plt.title('Correlation Matrix')
plt.show()
```

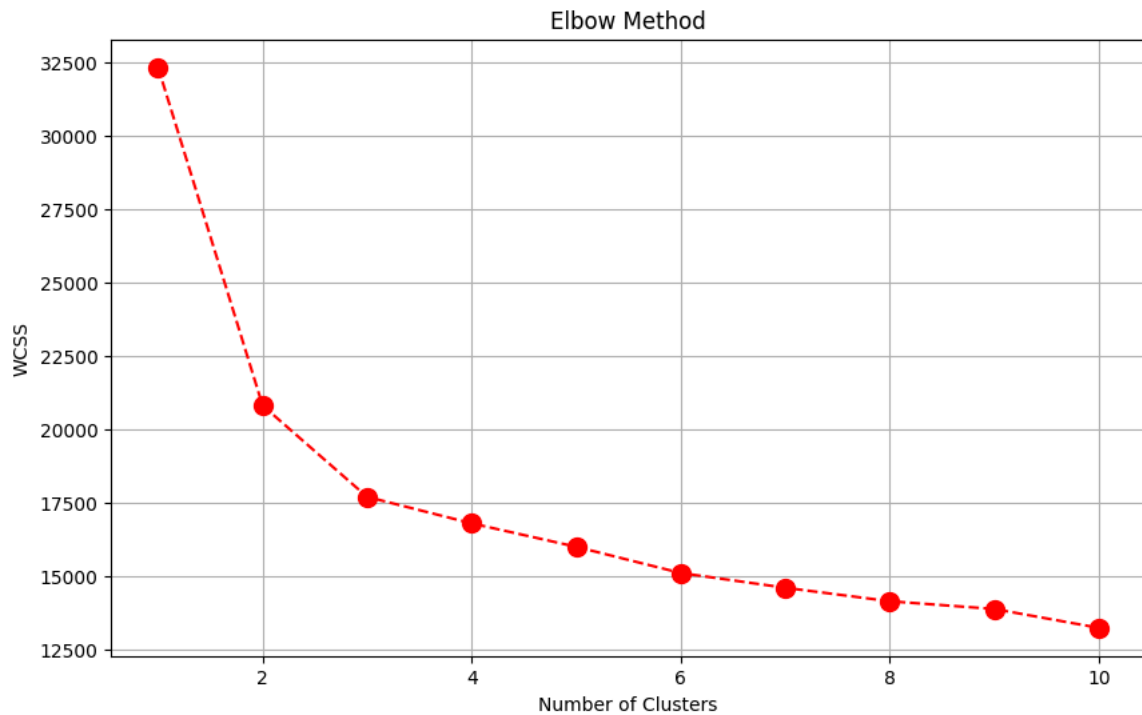


```
# Feature Selection for Segmentation
features = df[[
    'Income',
    'Kidhome',
    'Teenhome',
    'Recency',
    'MntWines',
    'MntFruits',
    'MntMeatProducts',
    'MntFishProducts',
    'MntSweetProducts',
    'MntGoldProds',
    'NumDealsPurchases',
    'NumWebPurchases',
    'NumCatalogPurchases',
    'NumStorePurchases',
    'NumWebVisitsMonth',
    'AcceptedCmp3',
    'AcceptedCmp4',
    'AcceptedCmp5',
    'AcceptedCmp1',
    'AcceptedCmp2',
    'Complain',
    'Z_CostContact',
    'Z_Revenue',
    'Response',
    'Age',
    'Customer_Days',
    'marital_Divorced',
    'marital_Married',
    'marital_Single',
    'marital_Together',
    'marital_Widow',
    'education_2n Cycle',
    'education_Basic',
    'education_Graduation',
    'education_Master',
    'education_PhD',
    'MntTotal',
    'MntRegularProds',
    'AcceptedCmpOverall',
    'TotalPurchases'
]]
```

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaled_features = scaler.fit_transform(features)
```

```
from sklearn.cluster import KMeans
# ind Optimal Number of Clusters
# elbow method
wcss = []
for i in range(1,11):
    kmeans = KMeans(n_clusters=i, init='k-means++', random_state=42)
    kmeans.fit(scaled_features)
    wcss.append(kmeans.inertia_)
plt.figure(figsize=(10,6))
plt.plot(range(1,11), wcss, marker='o', linestyle='--', color = 'red', markersize=10)
plt.grid(True)
```

```
plt.title('Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.show()
# Report Observation
# Usually elbow appears around 4 or 5 clusters.
```



```
kmeans = KMeans(n_clusters=4, init='k-means++', random_state=42)
df['Cluster'] = kmeans.fit_predict(scaled_features)
df['Cluster'].value_counts()
```

	count
Cluster	
0	830
2	437
3	417
1	337

dtype: int64

```
# Evaluate Clustering
from sklearn.metrics import silhouette_score
score = silhouette_score(scaled_features, df['Cluster'])
print(f'Silhouette Score: {score}')
```

Silhouette Score: 0.20238871757665244

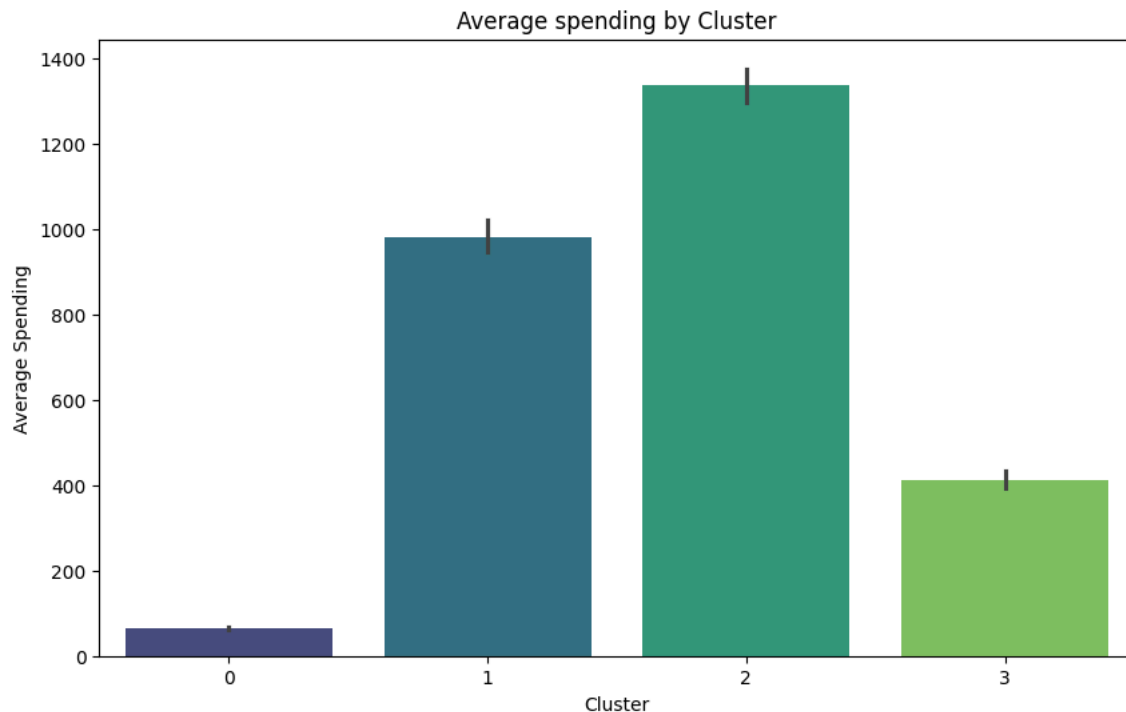
```
# Visualize Customer Segments
plt.figure(figsize=(10,6))
sns.scatterplot(data=df, x='Income', y='MntTotal', hue='Cluster', palette='viridis')
plt.title('Customer Segments')
plt.xlabel('Income')
plt.ylabel('Total Spending')
plt.show()
```




```
plt.figure(figsize=(10,6))
sns.barplot(data=df, x='Cluster', y='MntTotal', palette='viridis')
plt.xlabel('Cluster')
plt.ylabel('Average Spending')
plt.title('Average spending by Cluster')
plt.show()
```

/tmp/ipykernel_2580/719835090.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and
 sns.barplot(data=df, x='Cluster', y='MntTotal', palette='viridis')



```
# Segment Analysis
cluster_summary = df.groupby('Cluster')[[
    'Income',
    'Age',
    'MntTotal',
    'TotalPurchases'
]].mean()
cluster_summary
```

	Income	Age	MntTotal	TotalPurchases	
Cluster					
0	32702.144578	47.777108	64.284337	5.378313	
1	64646.501484	54.531157	981.089021	20.887240	
2	77033.496568	51.743707	1335.491991	19.144165	
3	52440.434053	54.350120	412.050360	13.261391	

Next steps:

[Generate code with cluster_summary](#)[New interactive sheet](#)

cluster_summary.round(2)

	Income	Age	MntTotal	TotalPurchases	
Cluster					
0	32702.14	47.78	64.28	5.38	
1	64646.50	54.53	981.09	20.89	
2	77033.50	51.74	1335.49	19.14	
3	52440.43	54.35	412.05	13.26	

```
# Insights & Recommendations
# Cluster 0 - High Value Customers
# High income
# High spending
# Frequent purchases

# Recommendation:
# VIP membership
# Exclusive offers
# Loyalty rewards

# Cluster 1 - Budget Customers
# Lower income
# Low spending

# Recommendation:
# Discount campaigns
# Seasonal promotions

# Cluster 2 - Potential Customers
# High income
# Moderate spending

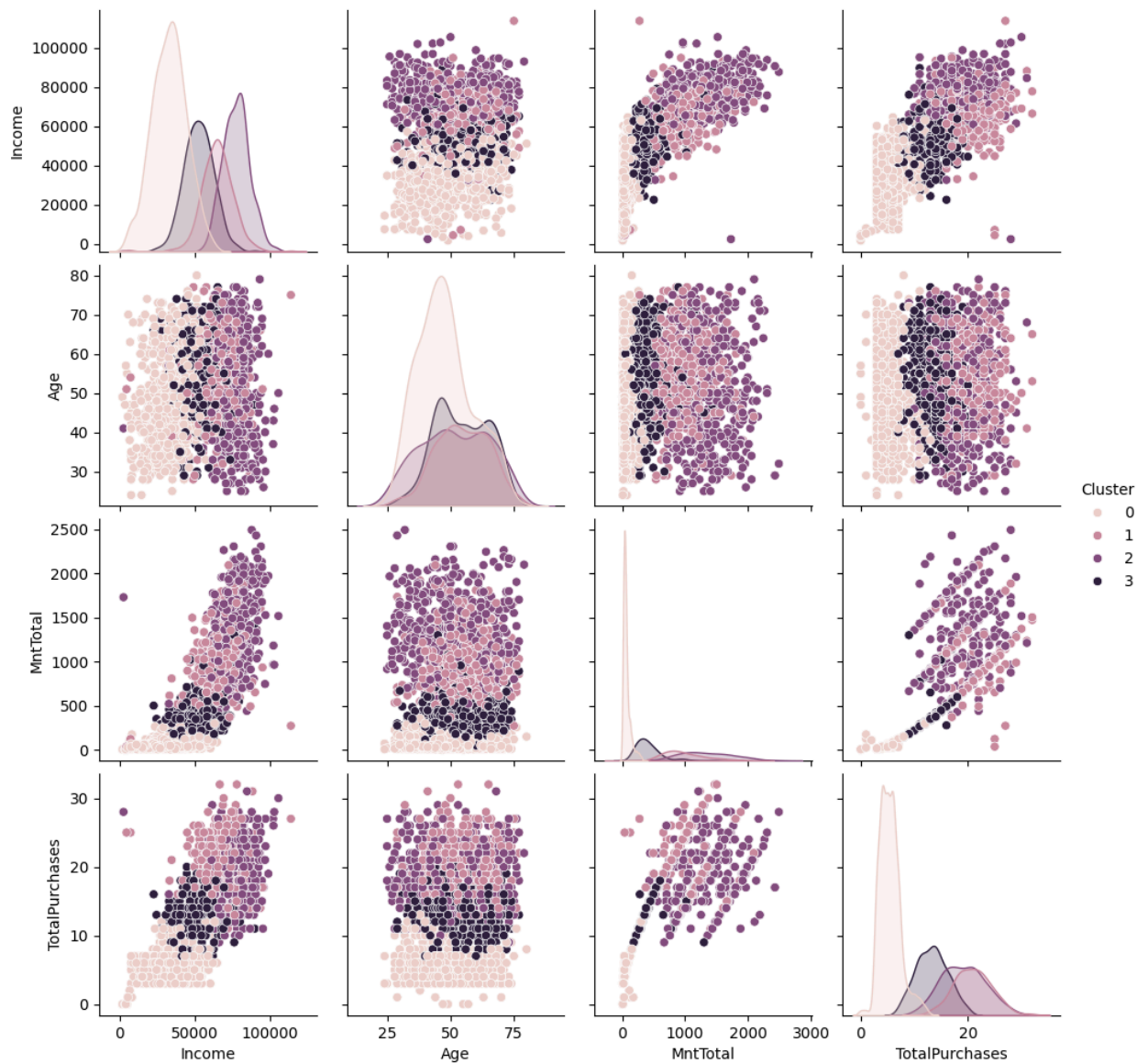
# Recommendation:
# Personalized recommendations
# Upselling opportunities

# Cluster 3 - Occasional Buyers
# Infrequent purchases
# Moderate spending

# Recommendation:
# Retargeting campaigns
# Email marketing
```

```
sns.pairplot(
    df[
        [
            'Income',
            'Age',
            'MntTotal',
            'TotalPurchases',
            'Cluster'
        ]
    ],
    hue='Cluster'
)

plt.show()
```



Customer Segmentation Results

The K-Means clustering algorithm successfully segmented customers into 4 distinct groups based on income, spending behavior, and purchase frequency.

The scatter plot of Income vs Total Spending clearly shows separate customer groups with different purchasing behaviors.

Segment Analysis

Cluster 0 - Low Value Customers

Characteristics

Lowest spending among all clusters

Low to medium income

Infrequent purchases

Limited engagement with products

Evidence

From the bar chart:

Average Spending ≈ 70

This is the lowest spending segment.

Business Recommendation

Offer discount coupons

Run promotional campaigns

Send personalized offers

Encourage first-time and repeat purchases

Cluster 1 - High Value Customers

Characteristics

High income

High spending

Frequent purchases

```
# Strong customer engagement
# Evidence
# Average Spending ≈ 980

# These customers generate significant revenue.

# Business Recommendation
# Introduce loyalty programs
# Provide premium memberships
# Offer exclusive product launches
# Prioritize customer retention
# Cluster 2 - Premium Customers
# Characteristics
# Highest income levels
# Highest spending levels
# Most profitable customer group
# Evidence
# Average Spending ≈ 1330

# This cluster has the highest average spending across all segments.

# Business Recommendation
# VIP rewards program
# Personalized recommendations
# Premium product bundles
# Early access to new products

# This is the most valuable customer segment.

# Cluster 3 - Moderate Customers
# Characteristics
# Moderate income
# Moderate spending
# Potential for growth
# Evidence
# Average Spending ≈ 410

# These customers spend more than Cluster 0 but significantly less than Clusters 1 and 2.

# Business Recommendation
# Upselling strategies
# Product recommendations
# Targeted email campaigns
# Seasonal offers
```